Ridge Regression on Diamonds Dataset

A Comparative Analysis of Optimization Algorithms

Davide Gamba
Matr. 1053470

University of Bergamo
Department of Management, Information and Production Engineering

March 30, 2026

Introduction

The objective of this study is to conduct a comparative analysis of the convergence behavior of various optimization algorithms applied on a Ridge Regression problem to the Diamonds Dataset.

In particular, the focus is on:

- Evaluating the respective algorithms convergence rates.
- Investigating how theoretical properties of the objective function (such as Convexity, Smoothness ecc..) have an impact to results.
- Evaluating how the L_2 regularization parameter (λ) of Ridge Regression influences algorithms behavior.

The following algorithms were implemented:

- Gradient Descent (Naive, Bounded, Smooth).
- Accelerated Gradient Descent (Nesterov's).
- Stochastic Gradient Descent (Naive, Strongly Convex)
- Newton's Method.
- Adagrad.

Dataset Overview: The Diamonds Dataset

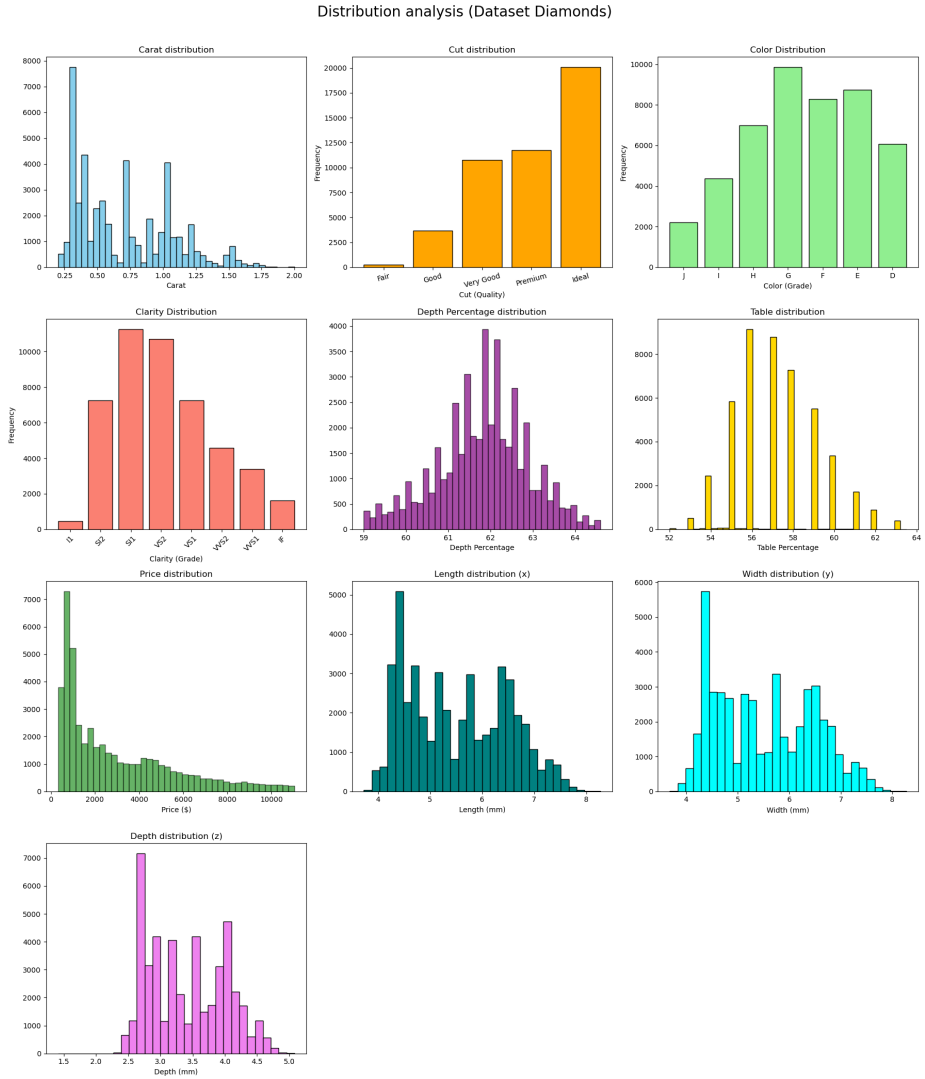
To evaluate the performance of the optimization algorithms, we utilized the **Diamonds Dataset**. The dataset consists of approximately **54,000 observations** ($n = 53.940$), each representing a diamond with **10 distinct attributes**.
The goal is to predict with a Ridge Regression the **Price** of a diamond based on its physical and quality characteristics.

Feature Description:

- **Target Variable (y):** `price` in US dollars.
- **Numerical Features:**
  - `carat`: Weight, the most significant predictor.
  - `x`, `y`, `z`: Length, Width, and Depth in mm.
  - `depth`: Total depth percentage ($z/\text{mean}(x, y)$).
  - `table`: Width of the top relative to the widest point.
- **Categorical Features (Ordinal):** `cut`, `color`, and `clarity`.

Dataset Overview: The Diamonds Dataset

Distribution of all the main variables within the diamonds dataset.



Dataset Characteristics: Multicollinearity

What is Multicollinearity?
It occurs when two or more independent variables in a regression model are highly linearly correlated, meaning one feature can be predicted from the others.

The Diamonds Dataset Case:
In our specific dataset, there is a strong correlation between `carat` (weight) and the physical dimensions (`x`, `y`, `z`). These features provide almost identical information.

Impact on Optimization:

- This structural redundancy results in a feature matrix X with a very small minimum eigenvalue ($\lambda_{\text{min}} \approx 0$).
- Consequently, the underlying optimization problem becomes highly **ill-conditioned**. It is particularly challenging for gradient-based methods because it creates flat valleys in the loss function landscape.

Ridge Regression

We address the **Ridge Regression** problem, which extends Ordinary Least Squares (OLS) by imposing an L_2 penalty on the model parameters to mitigate overfitting and multicollinearity.

The objective function is the sum of squared errors for each data point ($i = 1$ to n) plus the sum of squared weights (from $j = 1$ to d).

$$f(w) = \frac{1}{2n} \sum_{i=1}^{n} (y_i - x_i^T w)^2 + \frac{\lambda}{2} \sum_{j=1}^{d} w_j^2$$

The objective function $f(w)$ is defined in vector notation as follows.

$$f(w) = \underbrace{\frac{1}{2n} \|y - Xw\|^2}_{\text{Half MSE}} + \underbrace{\frac{\lambda}{2} \|w\|^2}_{\text{Regularization}}$$

Where:

- $X \in \mathbb{R}^{n \times d}$ is the feature matrix (includes a column of 1).
- $y \in \mathbb{R}^n$ is the target variable (price).
- $w \in \mathbb{R}^d$ is the weights vector.
- $\lambda > 0$ is the regularization hyperparameter.

Ridge Regression: Why the scaling?

Why the scaling?
Introducing scaling factors is convenient for mathematical and numerical stability:

- **The factor** $\frac{1}{2}$: Cancels out the exponent 2 when computing the gradient $\nabla f(w)$ and the derivative of $\|w\|^2$, simplifying to a linear form without constant multipliers.

- **The factor** $\frac{1}{n}$: Normalizes the error term with respect to the dataset size. This ensures that the magnitude of the loss is independent by the number of samples n. This allows the same learning rates to be effective regardless we use the full dataset or a smaller subset.

Gradient and Hessian Matrix

Since $f(w)$ is differentiable, we can compute the first-order derivative (the Gradient) and second-order derivative (the Hessian matrix) to minimize the objective function.

The Gradient $\nabla f(w)$
The gradient represents the steepest ascent direction of the function. To find the minimum, optimization algorithms like Gradient Descent move in the opposite direction of the gradient.
Let's expand the vector notation of the objective function:

$$f(w) = \frac{1}{2n}\|y - Xw\|^2 + \frac{\lambda}{2}\|w\|^2$$

$$f(w) = \frac{1}{2n}(y - Xw)^T(y - Xw) + \frac{\lambda}{2}w^T w$$

The Gradient $\nabla f(w)$

Let's expand the error quadratic term:

$$\begin{aligned}(y - Xw)^T(y - Xw) &= (y^T - (Xw)^T)(y - Xw) \\ &= (y^T - w^T X^T)(y - Xw) \\ &= y^T y - y^T Xw - w^T X^T y + w^T X^T Xw\end{aligned}$$

The terms $y^T Xw$ and $w^T X^T y$ are scalars (yielding a 1×1 result). In linear algebra, the transpose of a scalar is the scalar itself. So $(y^T Xw)^T = w^T X^T y$. Therefore, these two terms are identical and can be combined:

$$-y^T Xw - w^T X^T y = -2w^T X^T y$$

Now we substitute all in the original function:

$$f(w) = \frac{1}{2n}\left(y^T y - 2w^T X^T y + w^T X^T Xw\right) + \frac{\lambda}{2} w^T w$$

The Gradient $\nabla f(w)$

The gradient is the $d \times 1$ vector of partial derivatives respect to w. To compute the gradients and Hessians, we rely on the following standard matrix calculus properties (assuming w and b are column vectors, and A is a symmetric matrix):

1. **Constant rule:** $\nabla_w(b) = 0$
2. **Linear form:** $\nabla_w(w^T b) = \nabla_w(b^T w) = b$
3. **Jacobian of a linear mapping:** $\nabla_w(Aw) = A$
4. **Quadratic form:** $\nabla_w(w^T Aw) = 2Aw$ *(Requires A to be symmetric, i.e., $A = A^T$. In our case, $X^T X$ and I are both symmetric).*

Let's apply these rules to each term of $f(w)$:

1. **Term** $y^T y$: Does not depend on w.

$$\nabla_w(y^T y) = 0$$

2. **Term** $-2w^T(X^T y)$: Here $b = X^T y$.

$$\nabla_w \left(\frac{1}{2n}(-2w^T X^T y) \right) = -\frac{1}{n}\nabla_w(w^T(X^T y)) = -\frac{1}{n}X^T y$$

The Gradient $\nabla f(w)$

3. **Term** $w^T(X^TX)w$: Here $A = X^TX$. Note that X^TX is symmetric.

$$\nabla_w \left(\frac{1}{2n} w^T X^T X w \right) = \frac{1}{2n}(2X^TXw) = \frac{1}{n}X^TXw$$

4. **Regularization term:** $\frac{\lambda}{2}w^Tw$: Here $A = I$ (identity matrix).

$$\nabla_w \left(\frac{\lambda}{2} w^T I w \right) = \frac{\lambda}{2}(2Iw) = \lambda w$$

Gradient Result:
Summing all the parts together:

$$\nabla f(w) = \frac{1}{n}(X^TXw - X^Ty) + \lambda w$$

Factoring out common terms:

$$\nabla f(w) = \frac{1}{n}X^T(Xw - y) + \lambda w$$

The Hessian Matrix $\nabla^2 f(w)$

The Hessian matrix is the derivative of the gradient with respect to w (i.e., the second derivative of the original function). It is a $d \times d$ matrix.
Let's recall the expression for the gradient:

$$\nabla f(w) = \frac{1}{n} X^T X w - \frac{1}{n} X^T y + \lambda w$$

We differentiate each term with respect to w. We use the matrix calculus identity $\nabla_w (Aw) = A$:

1. **Term** $\frac{1}{n} X^T X w$: The constant matrix multiplying w is $\frac{1}{n} X^T X$.

$$\nabla_w \left(\frac{1}{n} X^T X w \right) = \frac{1}{n} X^T X$$

The Hessian Matrix $\nabla^2 f(w)$

2. **Term** $-\frac{1}{n}X^T y$: This term does not contain w (it is a constant vector).

$$\nabla_w \left(-\frac{1}{n} X^T y \right) = 0$$

3. **Term** λw: We can rewrite this as $\lambda I w$. The constant matrix is λI.

$$\nabla_w (\lambda I w) = \lambda I$$

We put them together, so the **Hessian Matrix** is :

$$H = \nabla^2 f(w) = \frac{1}{n} X^T X + \lambda I$$

$f(w)$ is strictly convex

To investigate the convexity of our objective function, we rely on the formal definition of definite matrices based on their quadratic forms.

A symmetric matrix M is Positive Semi-Definite if $v^T Mv \geq 0$ for all vectors v, and Positive Definite if $v^T Mv > 0$ for all non-zero vectors $v \neq 0$.

Let's apply this definition to our Ridge Regression Hessian matrix, $H = \frac{1}{n} X^T X + \lambda I$, by analyzing its quadratic form $v^T Hv$ for any arbitrary non-zero vector $v \in \mathbb{R}^d$:

$$v^T Hv = v^T \left(\frac{1}{n} X^T X + \lambda I \right) v$$

By distributing the multiplication, we get:

$$v^T Hv = \frac{1}{n} v^T X^T Xv + \lambda v^T Iv$$

$f(w)$ is strictly convex

We can rewrite the first term using the property of transposes, $(Xv)^T = v^T X^T$, and the second term using the definition of the squared L_2 norm, $v^T v = \|v\|_2^2$:

$$v^T H v = \frac{1}{n}(Xv)^T(Xv) + \lambda\|v\|_2^2$$

$$v^T H v = \underbrace{\frac{1}{n}\|Xv\|^2}_{\text{OLS Base}} + \underbrace{\lambda\|v\|^2}_{\text{Ridge Penalty}}$$

Now, let's evaluate the two components:

- **The OLS Base:** The squared norm $\|Xv\|^2$ is a squared length, meaning it is always non-negative (≥ 0). This proves that the OLS Hessian ($\frac{1}{n}X^T X$) is intrinsically **Positive Semi-Definite**. However, if features are perfectly collinear, there exist non-zero vectors v where $Xv = 0$. In these cases, the quadratic form equals zero, meaning the loss function has flat valleys with no unique minimum.

$f(w)$ is strictly convex

- **The Ridge Penalty:** For any non-zero vector $v \neq 0$, its squared norm $\|v\|^2$ is strictly positive. Since our regularization hyperparameter is strictly positive ($\lambda > 0$), the entire term $\lambda\|v\|^2$ must be strictly greater than zero (> 0).

Because we are adding a strictly positive term ($\lambda\|v\|^2 > 0$) to a non-negative term ($\frac{1}{n}\|Xv\|^2 \geq 0$), the entire sum is guaranteed to be strictly positive:

$$v^T Hv > 0 \quad \text{for all } v \neq 0$$

This proves that the Ridge Hessian matrix H is **Positive Definite**. The L_2 penalty mathematically eliminates any flat regions, guaranteeing that the objective function possesses a single, unique global minimum, so that means $f(w)$ is strictly convex.

Ridge Regression - Properties

To understand how iterative optimization algorithms will behave on Ridge Regression objective function $f(w)$, we must analyze its properties.

1. Convexity: Yes (Strictly Convex)
As proven in the previous section using the Hessian's quadratic form, the addition of the L_2 penalty ensures that the Hessian matrix H is Positive Definite. Therefore, the function is not just convex, but **strictly convex**. This guarantees that any local minimum is the unique global minimum w^*.

2. Lipschitz Continuous: No (Globally)
A function $f(w)$ is globally B-Lipschitz if its gradients are bounded in magnitude everywhere ($\|\nabla f(w)\| \leq B$). Our objective function $f(w)$ is a quadratic function. As the weights w can grow toward infinity, the gradient $\nabla f(w) = \frac{1}{n} X^T(Xw - y) + \lambda w$ also can grows linearly toward infinity. Because the gradient is unbounded over the entire domain $\mathbb{R}^d$, the function itself is **not globally Lipschitz continuous**.

Ridge Regression - Properties

3. Smoothness (L-smooth): Yes
It is L-**Smooth**. The curvature does not change abruptly; the gradient doesn't oscillate wildly. The function always lies *below* a certain parabola:

$$f(y) \leq f(x) + \nabla f(w)^T (y - x) + \frac{L}{2}\|x - y\|^2 \quad \forall x, y$$

- **Value of L:** It corresponds to the largest eigenvalue of the Hessian matrix (the maximum curvature of the loss landscape).

$$L = \lambda_{\max}\left(\frac{1}{n} X^T X\right) + \lambda$$

- **Consequence:** This value acts as the absolute **speed limit** for Gradient Descent. The optimal safe step size is $\gamma = \frac{1}{L}$.

Ridge Regression - Properties

4. Strong Convexity (μ-strongly convex): Yes
It is μ-**Strongly Convex**. The curvature is "at least" μ in every direction. The function always lies *above* the quadratic function:

$$f(y) \geq f(x) + \nabla f(x)^T (y - w) + \frac{\mu}{2}\|x - y\|^2 \quad \forall x, y$$

- **Value of μ:** It corresponds to the smallest eigenvalue of the Hessian matrix.

$$\mu = \lambda_{\min}\left(\frac{1}{n}X^T X\right) + \lambda$$

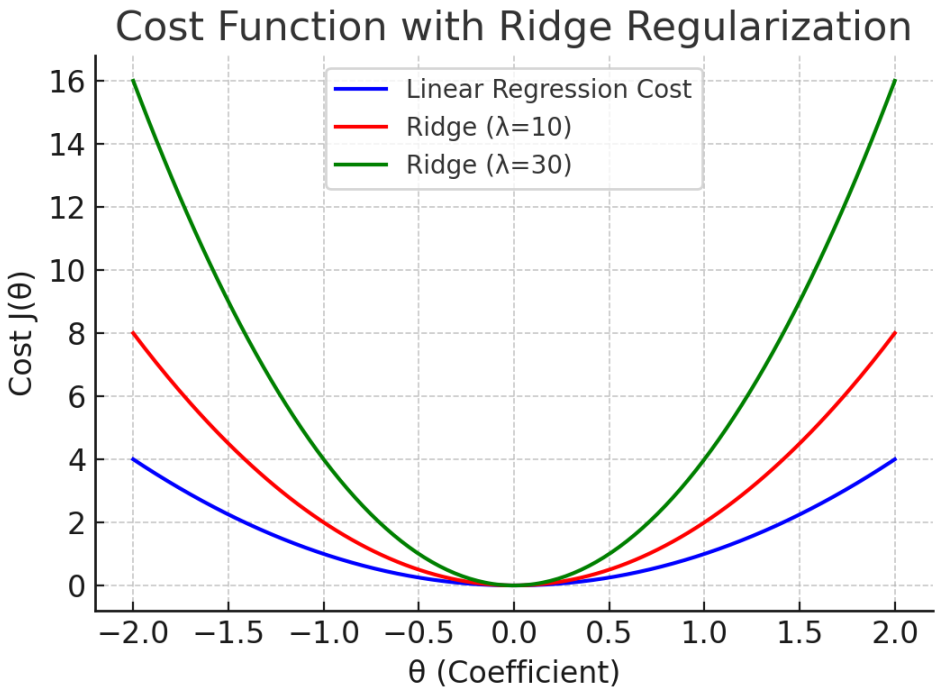
If the feature matrix X is not full-rank, for instance, due to perfectly collinear features, the minimum eigenvalue of the OLS component $\frac{1}{n}X^T X$ is exactly 0, meaning $\mu = \lambda$. This mathematically proves that without regularization ($\lambda = 0$), the problem might not be strongly convex, which would make convergence much slower.

- **Consequence:** The larger μ is, faster the algorithm will converge to the minimum.

The expected effect of L2 regularization

We expect the regularization parameter λ alter the geometry of our cost function.

Specifically, increasing λ directly increases μ, the **strong convexity parameter**, which dictates the *minimum curvature* of the loss landscape. By adding the quadratic penalty $\frac{\lambda}{2}\|w\|_2^2$, we are guaranteeing that the function curves upwards by at least a factor of λ in every single direction. This means that any "flat valleys" caused by multicollinearity are physically eliminated, making the objective function narrower.



The expected effect of L2 regularization

Expected Results on Convergence:

Because the minimum curvature (μ) is strictly bounded away from zero and increases with λ, the loss function no longer has any flat regions where the algorithm might stall.

The gradients will always provide a strong and clear signal pointing toward the minimum. Consequently, we expect that as λ increases, the Gradient Descent algorithm will be pushed toward the global minimum much more rapidly, requiring significantly fewer iterations to reach the optimal solution.

Dataset pre-processing steps

To prepare the dataset for our optimization algorithms, we now apply the following preprocessing steps:

1. **Ordinal Encoding:** Categorical variables (`cut`, `color`, `clarity`) were mapped to integer values, preserving their inherent hierarchy. This safely converts text labels into a numerical format that the mathematical model can process.

2. **Removing Outliers:** We filter out extreme anomalous data points. Since our objective function relies on Mean Squared Error (MSE), it is highly sensitive to outliers, which could disproportionately skew the optimization of our weights.

Before feeding our data into the optimization algorithm, we must carefully preprocess our matrices.

Dataset pre-processing steps

1. **Standardizing the Features (X) and the Target (y):** By subtracting the mean and dividing by the standard deviation, we ensure all features and the target share the same scale (a mean of 0 and a variance of 1).
   Mathematically, for each feature column x_j in X and for the target vector y, the transformation is defined as:

$$x_j = \frac{x_j - \mu_j}{\sigma_j} \qquad y = \frac{y - \mu_y}{\sigma_y}$$

   This is crucial because it prevents features with naturally larger ranges from dominating the cost function. Geometrically, it transforms the loss landscape into a much more spherical shape, which allows Gradient Descent to point much more directly toward the minimum and converge significantly faster.

2. **The Bias Trick (Adding a column of 1s):** We append a column of ones to our feature matrix X. This mathematical trick allows us to absorb the intercept (or bias) term directly into our weight vector w.

Experimental Approach and Goals

We will now test and compare the various optimization algorithms. Each algorithm will be evaluated across a predefined set of regularization hyperparameters: $\lambda \in \{10^{-5}, 10^{-4}, 10^{-3}, 0.01, 0.1, 1.0\}$.

The objective of this experiment is to conduct comparative analysis with a focus on:

- **Impact of λ on convergence:** On the same algorithm, how varying the λ penalty impacts the number of iterations and time required to reach convergence.
- **Compare different algorithms** How different algorithms (GD, SGD, Newton method ecc..) performs under the same regularization conditions.

Exact Analytical Solution

We compute the exact global minimum w^* and its corresponding optimal loss $f(w^*)$ across the predefined set of regularization hyperparameters $\lambda \in \{10^{-5}, 10^{-4}, 10^{-3}, 0.01, 0.1, 1.0\}$. By knowing the absolute minimum w^* beforehand, we can use it later as a benchmark to understand when our iterative algorithms converge to the true solution.

To find this baseline, we directly apply the **closed-form solution** using `np.linalg.solve(A, b)` which efficiently and stably solves the underlying linear system of equations

$$\left(\frac{1}{n}X^T X + \lambda I\right) w = \frac{1}{n}X^T y$$

providing us with a accurate weight vector so we can compute the optimal values for each λ.

Exact Analytical Solution

λ	10^{-5}	10^{-4}	10^{-3}	0.01	0.1	1.0
$f(w^*)$	0.04265771	0.04276069	0.04369175	0.04977553	0.07101217	0.16355421

As the regularization hyperparameter λ increases, the training error (loss) increases as well. This happens because the optimization algorithm is no longer solely focused on minimizing the residual errors to fit the training data perfectly.

Instead, it must balance that goal with keeping the model weights (w) as small as possible to minimize the heavy L_2 penalty. By forcing the weights to shrink, we restrict the model's flexibility, preventing it from capturing every minor fluctuation and noise in the training set.

Gradient Descent

Now we implement the core iterative optimization algorithm: **Gradient Descent**.

Gradient descent by definition updates the weight vector w step-by-step by moving in the opposite direction of the gradient, scaled by the learning rate (step size) γ:

$$w^{(t+1)} = w^{(t)} - \gamma \nabla f(w^{(t)})$$

We have introduced a **stopping criterion**. Rather than running for `max_iters`, the algorithm continuously monitors ϵ (epsilon): the difference between the current loss and the exact analytical minimum w^* we established in the previous step.

If this difference drops below a microscopic tolerance threshold (`tol = 1e-6`), the algorithm recognizes that it has practically reached the global minimum and triggers an "early stop".

$$\epsilon = f(w_T) - f(w^*)$$

Gradient Descent - Naive Learning Rate

We execute our Gradient Descent algorithm across our predefined set of λ values.

For this first run, we are using a **naive, very small learning rate $\gamma = 0.0001$** .

Choosing a tiny step size is a conservative and highly stable approach: it guarantees that our algorithm will not overshoot the minimum or diverge.

However, this safety comes with a heavy trade-off in efficiency. Taking such microscopic steps means the algorithm might require a massive number of iterations to finally reach the optimal solution, which is why we set a high computational budget (`max_iters = 50000`).

We apply Naive Gradient Descent five times, one for each Ridge Regression regularization λ problem defined previously, and store the metrics: final loss, iterations, and execution time.

Gradient Descent - Naive Learning Rate

Final results per λ using a naive learning rate ($\gamma = 0.0001$):

λ	**Loss**	γ	**Iters**	**Time[s]**
10^{-5}	0.05510	0.0001	50000	9.55
10^{-4}	0.05512	0.0001	50000	10.53
10^{-3}	0.05528	0.0001	50000	9.70
0.01	0.05689	0.0001	50000	10.24
0.1	0.07171	0.0001	50000	11.27
1.0	0.16356	0.0001	31209	6.40

Observations:

- For low regularizations (from $\lambda = 10^{-5}$ to 0.1), the algorithm hits the `max_iters` limit of 50.000 without reaching the tolerance of the early stop.
- Only for the strongest regularization ($\lambda = 1.0$), the algorithm is pushed fast enough to converge in exactly 31.209 iterations.

Gradient Descent - Assuming Bounded Gradient

Assume the distance between initial solution and optimal solution is bounded by R and this bounded region contains all iterates. Since we initialize our initial solution vector to **Zero** (`np.zeros`), the distance coincides exactly with the norm of the solution:

$$\|w_0 - w^*\| \leq R$$

$$\|0 - w^*\| = \|w^*\| \leq R$$

Basically the assumption on R means: *"The optimal solution and the optimization path to reach it lie within a sphere of radius R."*

- Since we have **standardized the data** (X and y have zero mean and unit variance), the optimized weights w will naturally be small (typically between -1 and 1)
- We set $R = 5.0$. A generous safety margin (a "Bounded Region") that is virtually guaranteed to contain the true solution.

Gradient Descent - Assuming Bounded Gradient

We need to find a number B such that $\|\nabla f(w)\| \leq B$ for every w within the radius R. We rewrite the gradient formula in this way:

$$\nabla f(w) = \left(\frac{1}{n} X^T X + \lambda I\right) w - \frac{1}{n} X^T y$$

Using the triangle inequality ($\|a + (-b)\| \leq \|a\| + \| - b\|$) and the definition of matrix norms:

$$\|\nabla f(w)\| \leq \underbrace{\left\|\frac{1}{n} X^T X + \lambda I\right\|}_{\text{This is smooth parameter L}} \cdot \underbrace{\|w\|}_{\leq R} + \underbrace{\left\|\frac{1}{n} X^T y\right\|}_{\text{Constant}}$$

So the formula for B is:

$$B = L \cdot R + \left\|\frac{1}{n} X^T y\right\|$$

Gradient Descent - Assuming Bounded Gradient

By making the assumption that our optimization path remains within a bounded region $\|w_0 - w^*\| \leq R$, and having calculated the maximum gradient bound B such that $\|\nabla f(w)\| \leq B$ for all w, we have effectively established that our objective function is B-**Lipschitz continuous** within this domain.

For Lipschitz continuous functions, optimization theory provides a specific, optimal step size: we can now set γ using the formula:

$$\gamma = \frac{R}{B\sqrt{T}}$$

where $T = 10.000$ iterations.

Gradient Descent - Assuming Bounded Gradient

Final results per λ using the bounded gradient step size (γ):

λ	**Loss**	γ	**Iters**	**Time[s]**
10^{-5}	0.04966	0.00214	10000	1.86
10^{-4}	0.04969	0.00214	10000	2.04
10^{-3}	0.04997	0.00214	10000	2.88
0.01	0.05263	0.00214	10000	2.15
0.1	0.07103	0.00210	10000	2.02
1.0	0.16356	0.00176	1768	0.32

Observations:

- **Larger Steps:** The dynamic $\gamma \approx 0.002$ is safely 20x larger than the naive guess.
- **Massive Speedup:** For $\lambda = 1.0$, it converges in only 1,768 iterations (vs. 31,209).
- **Higher Accuracy:** It reaches a strictly lower loss in just 10,000 steps compared to the 50,000 steps of the naive approach.

Gradient Descent - Smoothness

Previously we have seen that function is L-**Smooth**. The curvature does not change abruptly; the gradient does not fluctuate wildly. The function always stays "below" a certain bounding parabola.

- **Value of L:** It is the largest eigenvalue of the Hessian matrix (the maximum curvature).

$$L = \lambda_{\max}\left(\frac{1}{n}X^T X\right) + \lambda$$

or equivalently

$$L = \left\|\frac{1}{n}X^T X\right\| + \lambda$$

- **Consequence:** This value is the **speed limit** for Gradient Descent. If we use a learning rate $\gamma > \frac{2}{L}$, the algorithm will diverge. The optimal safe step size is $1/L$.

Gradient Descent - Smoothness

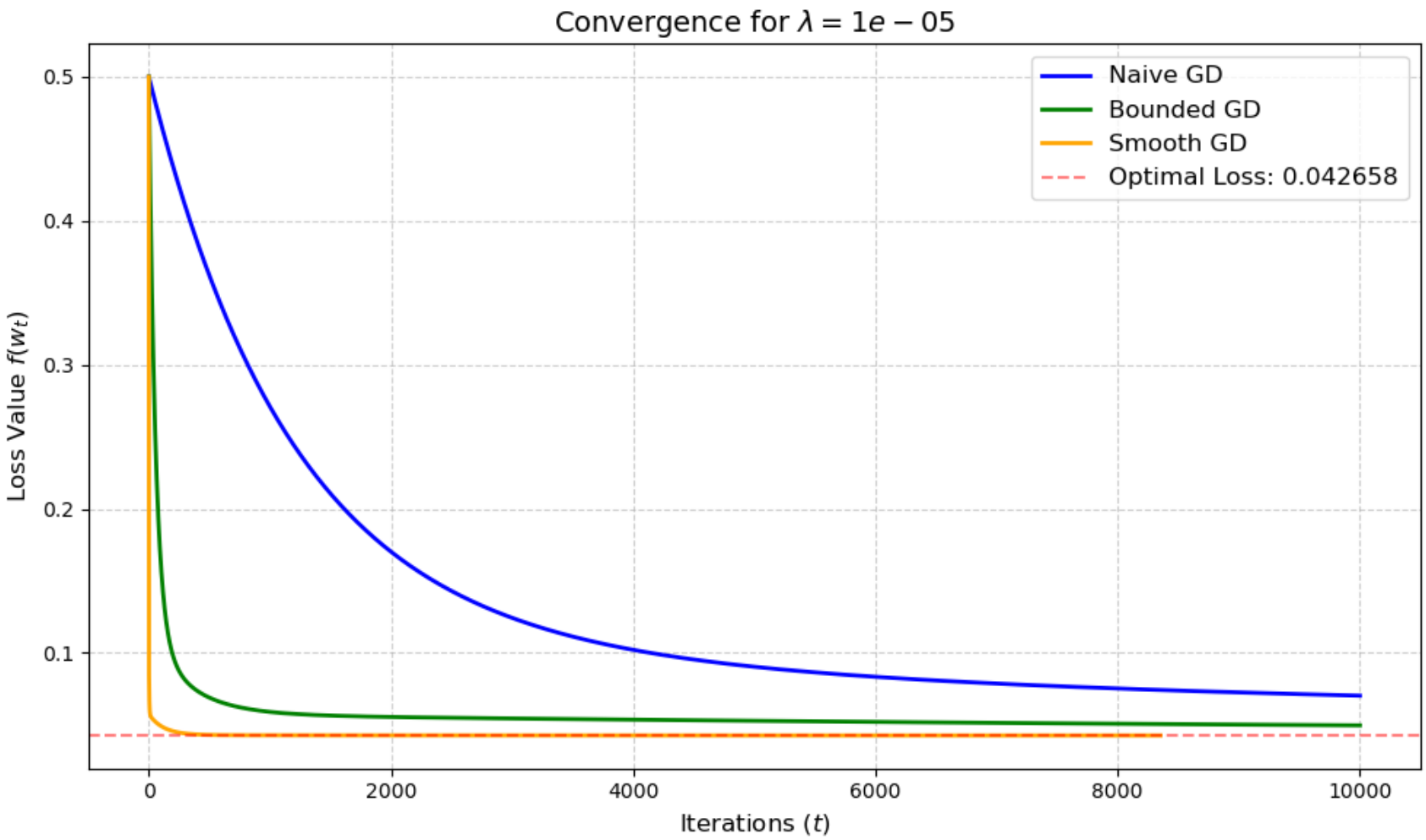
Final results per λ using the optimal step size ($\gamma = 1/L$):

λ	**Loss**	$\gamma = 1/L$	**Iters**	**Time[s]**
10^{-5}	0.04266	0.23247	8353	1.56
10^{-4}	0.04276	0.23247	7767	1.74
10^{-3}	0.04369	0.23242	4438	1.76
0.01	0.04978	0.23193	765	0.22
0.1	0.07101	0.22719	142	0.07
1.0	0.16355	0.18862	15	0.01

Observations:

- **Optimal Steps:** The safe learning rate ($\gamma \approx 0.23$) is $\sim$2,300x larger than our naive guess.
- **Lightning Fast:** For $\lambda = 1.0$, it perfectly converges in just 15 iterations (down from 31.209).
- **Perfect convergence:** For all case the algorithm converge to optimal solution.

Convergence Analysis: Comparing Gradient Descent implementations



Nesterov's Accelerated Gradient Descent

Now we implement the **Accelerated Gradient Descent** algorithm. This method improves standard gradient descent by computing the next iteration point as a dynamically weighted average between a conservative "smooth" step and a more "aggressive" step.

Mathematically, choosing arbitrary initial points $\mathbf{z}_0 = \mathbf{u}_0 = \mathbf{w}_0$, for $t \geq 0$ we set:

$$\mathbf{u}_{t+1} := \mathbf{w}_t - \frac{1}{L}\nabla f(\mathbf{w}_t)$$

$$\mathbf{z}_{t+1} := \mathbf{z}_t - \frac{t+1}{2L}\nabla f(\mathbf{w}_t)$$

$$\mathbf{w}_{t+1} := \frac{t+1}{t+3}\mathbf{u}_{t+1} + \frac{2}{t+3}\mathbf{z}_{t+1}$$

- `w` represents the main weight vector.
- `u` represents the prudent/smooth update.
- `z` represents the aggressive update.

Nesterov's Accelerated Gradient Descent

Following the same testing framework used previously: we loop through our predefined `lambda_set`.

For each λ, we calculate the exact L-smoothness parameter to scale our steps correctly. The algorithm logs the weights and execution time, and features an early stopping mechanism (`tol=1e-6`) that halts execution once the loss successfully converges to our pre-calculated theoretical optimum (`opt_value`).

Nesterov's Accelerated Gradient Descent

Final results per λ using Accelerated Gradient Descent:

λ	**Loss**	**Iters**	**Time[s]**
10^{-5}	0.04266	203	0.10
10^{-4}	0.04276	202	0.12
10^{-3}	0.04369	152	0.07
0.01	0.04978	86	0.05
0.1	0.07101	23	0.01
1.0	0.16355	12	0.005

Observations:

- **Massive Speedup:** For ill-conditioned problems ($\lambda = 10^{-5}$), iterations go down from 8,353 to just 203 ($\sim$40x improvement).
- **Faster Convergence Rate:** Validates the theoretical leap from $O(1/T)$ to $O(1/T^2)$.
- **Diminishing Returns:** For $\lambda = 1.0$ (a steep, well-conditioned bowl), the advantage is marginal (12 vs. 15 iterations).

Stochastic Gradient Descent with Mini-batches

In standard Gradient Descent, computing the exact gradient requires evaluating the entire dataset (all n samples) at every single step. While highly accurate, this becomes computationally prohibitive and slow for massive datasets. **Mini-batch Stochastic Gradient Descent (SGD)** addresses this bottleneck by approximating the true gradient using a much smaller, randomly selected subset of the data of size m (where $m < n$).

Mathematically, starting from an initial point $\mathbf{x}_0 \in \mathbb{R}^d$, at each iteration t we choose a random subset of indices $\{h_1, \ldots, h_m\} \subseteq \{1, \ldots, N\}$ and update the weights using a learning rate $\gamma_t \geq 0$:

$$\mathbf{w}_{t+1} := \mathbf{w}_t - \gamma_t \frac{1}{m} \sum_{j=1}^{m} \nabla f_{h_j}(\mathbf{w}_t)$$

Stochastic Gradient Descent with Mini-batches

To realize the random subset selection, the algorithm operates in "epochs". At the start of an epoch, the entire dataset is randomly shuffled (`np.random.permutation`). Within an epoch, a loop then sequentially iterates over this shuffled data in chunks of size `batch_size` (our m).

The `minibatch_gradient` function then calculates the scaled gradient on this small chunk, and the main loop updates the weights.

Just as we did in our initial experiments with standard Gradient Descent, we will establish a baseline by using a simple, "naive" constant learning rate of $\gamma = 10^{-4}$, with a `batch_size` $= 1000$.

Stochastic Gradient Descent with Mini-batches

Final results per λ using Stochastic Gradient Descent with a naive learning rate ($\gamma = 0.0001$):

λ	**Loss**	γ	**Iters**	**Time[s]**
10^{-5}	0.05510	0.0001	50000	12.79
10^{-4}	0.05512	0.0001	50000	13.10
10^{-3}	0.05528	0.0001	50000	13.14
0.01	0.05689	0.0001	50000	12.84
0.1	0.07171	0.0001	50000	13.54
1.0	0.16356	0.0001	31254	7.35

Observations:

- Results are almost identical to our initial Naive Full-Batch Gradient Descent. The algorithm still hits the hard limit of 50,000 iterations for small λ values, getting stuck at a suboptimal loss of roughly 0.05510 instead of reaching the true minimum (≈ 0.04266).

Mini batches SGD - Decaying Learning Rate

Now try doing a mini-batch SGD with a better learning rate using the fact that the objective function is μ-strongly convex. From Tame Strong convexity, the learning rate to use is:

$$\gamma_t = \frac{2}{\mu(t+1)}$$

and it decays over each iteration.

However, applying this formula naively when our regularization parameter λ is small leads to immediate divergence.

- If μ is very small, the initial step size γ_t becomes very large.
- This massively exceeds our theoretical L-smoothness "speed limit" $(2/L)$, causing the algorithm to diverge.

Mini batches SGD - Decaying Learning Rate

Looking closely at our calculated strong convexity parameters (μ), we can demonstrate why applying the classic theoretical decay rate $\gamma_t = \frac{2}{\mu(t+1)}$ causes divergence.

For our smallest ridge penalization ($\lambda = 10^{-5}$), the minimum curvature of our loss function is extremely flat, resulting in $\mu \approx 0.00130$. If we plug this value into the pure "Tame Strong Convexity" formula to calculate the learning rate for our very first iteration ($t = 0$), we get:

$$\gamma_0 = \frac{2}{0.00130} \approx 1538$$

This initial step size is catastrophically large. As we established earlier, the absolute maximum safe step size before an algorithm diverges is bounded by the smoothness parameter $(2/L) \approx 0.46$. An initial step of 1538 massively violates this theoretical "speed limit".

This same issue occurs across all other small values of the ridge regularization parameter λ.

Mini batches SGD - Decaying Learning Rate

To safely apply the optimal decay rate by the Tame Strong Convexity, we propose an adjusted learning rate:

$$\gamma_t = \frac{2}{L + \mu(t+1)}$$

This cover two fundamental concepts in optimization theory:

- **Initial Stability ($t = 0$):** At the very first iteration, the learning rate evaluates to exactly $\frac{2}{L+\mu} < \frac{2}{L}$. This perfectly respect the theoretical optimal speed limit for smooth strongly convex functions, acting as a safety cap that prevents the algorithm from diverging, regardless of how small μ is.

- **Asymptotic Convergence ($t \to \infty$):** As the iterations progress, the L term becomes dominated by μt. The learning rate transitions to approximate $\frac{2}{\mu t}$, strictly adhering to the $O(1/t)$ decay rate required to suppress the stochastic noise of the mini-batches and guarantee convergence to the global minimum.

Mini batches SGD - Decaying Learning Rate

Final results per λ using the adjusted decaying learning rate (γ_t):

λ	**Loss**	**Iters**	**Time[s]**
10^{-5}	0.04266	14199	3.58
10^{-4}	0.04276	15115	3.94
10^{-3}	0.04369	8104	2.23
0.01	0.04978	1730	0.71
0.1	0.07101	404	0.13
1.0	0.16355	101	0.04

Observations:

- **Guaranteed Convergence:** Safely bounded initial steps eliminate chaotic bouncing, achieving perfect convergence to the exact global minimum for every λ.
- **Variance Trade-off:** For ill-conditioned problems ($\lambda = 10^{-5}$), SGD is actually slower than full-batch GD ($\sim$14k vs. 8.3k iterations).
- **Noise Reduction:** To settle exactly at the minimum, the decaying learning rate must become extremely small, forcing thousands of tiny, cautious steps to average out mini-batch noise.

Newton's Method

Until now, we have relied on first-order optimization algorithms (like Gradient Descent and its variants), which only use the gradient (the slope) to determine the direction of our next step. **Newton's Method** introduces a massive upgrade by incorporating second-order derivative information—specifically, the curvature of the loss landscape represented by the Hessian matrix $\nabla^2 f(\mathbf{x})$.

By understanding both the slope and the curvature, the algorithm doesn't just step blindly down the hill; it models the exact shape of the convex bowl and points directly toward the bottom.

Mathematically, for minimizing a convex function $f : \mathbb{R}^d \to \mathbb{R}$ in the d-dimensional case, the update rule is defined as:

$$\mathbf{w}_{t+1} := \mathbf{w}_t - \nabla^2 f(\mathbf{w}_t)^{-1} \nabla f(\mathbf{w}_t)$$

Newton's Method

Final results per λ using Newton's Method:

λ	**Loss**	**Iters**	**Time[s]**
10^{-5}	0.04266	1	0.006
10^{-4}	0.04276	1	0.004
10^{-3}	0.04369	1	0.003
0.01	0.04978	1	0.003
0.1	0.07101	1	0.003
1.0	0.16355	1	0.002

Newton's Method

Looking at the experimental results, **Newton's Method achieves exact numerical convergence (to our 10^{-6} tolerance) in a single update iteration.**

This result is a direct consequence of the underlying mathematical nature of **Ridge Regression**. The objective function is an **entirely quadratic** and strictly convex function with respect to the weight vector w.

$$f(w) = \frac{1}{2n}\|Xw - y\|^2 + \frac{\lambda}{2}\|w\|^2$$

The One-Step Convergence Lemma:
On nondegenerate quadratic functions, with any arbitrary starting point $w_0 \in \mathbb{R}^d$, Newton's method yields the exact global minimum w^* in exactly one step ($w_1 = w^*$).

Because the Ridge Regression objective function satisfies the conditions of a nondegenerate quadratic function (because a matrix with strictly positive eigenvalues is guaranteed to be strictly positive definite and invertible), this experimental behavior aligns exactly with optimization theory.

Adagrad

AdaGrad modifies standard Stochastic Gradient Descent by introducing a dynamic, per-coordinate learning rate. At each iteration t, we first pick a stochastic gradient $\mathbf{g}_t$.

1. Gradient Accumulation

We accumulate the history of squared gradients for each individual coordinate i:

$$[G_t]_i := \sum_{s=0}^{t} ([\mathbf{g}_s]_i)^2 \quad \forall i$$

(This step tracks how strong the gradients have been in the past specifically for the i-th coordinate).

2. Parameter Update

We then update each coordinate i of our parameter vector $\mathbf{w}$:

$$[\mathbf{w}_{t+1}]_i := [\mathbf{w}_t]_i - \frac{\gamma}{\sqrt{[G_t]_i}} [\mathbf{g}_t]_i \quad \forall i$$

Adagrad

The Adaptive Learning Rate:
The crucial part of this formula is the term $\frac{\gamma}{\sqrt{[G_t]_i}}$, which acts as our new coordinate-specific learning rate.

- By dividing the base learning rate γ by the square root of the accumulated gradients, the algorithm automatically **reduces the step size** for coordinates that have historically had large gradients.
- Instead, if a coordinate has had consistently small gradients, its learning rate remains relatively **higher**.

This mechanism speeds up convergence and drastically reduces wild oscillations along directions where gradients are already large.

Adagrad

Final results per λ using the Adagrad algorithm with $\gamma = 0.1$:

λ	**Loss**	**Iters**	**Time[s]**
10^{-5}	0.04266	42511	11.99
10^{-4}	0.04276	37110	9.61
10^{-3}	0.04369	19376	4.73
0.01	0.04978	3477	0.78
0.1	0.07101	9059	1.98
1.0	0.16356	8292	1.92

Adagrad

- **Guaranteed Convergence:** Adagrad successfully reached the exact global minimum for all λ values. It automatically scaled its own steps, completely avoiding the need to compute theoretical parameters like Lipschitz smoothness (L) or strong convexity (μ).

- **Slower than Adjusted SGD:** The algorithm is significantly slower than our tuned SGD (taking $\sim$42k iterations for $\lambda = 10^{-5}$).

- **High Curvature Penalty:** Stronger regularization (e.g., $\lambda = 1.0$) creates a steeper convex bowl. This produces massive gradients in the very first iterations, which permanently push up the denominator G_t. The algorithm push down γ too early, requiring thousands of tiny, redundant steps to cross the final tolerance threshold.

Summary of Results for $\lambda = 10^{-5}$

Performance comparison on the ill-conditioned problem ($\lambda = 10^{-5}$):

Optimization Method	**Loss**	**Iterations**	**Time[s]**
Naive Gradient Descent	0.05510	50000*	∼10.50
Bounded Gradient Descent	0.04966	10000*	1.86
Smooth Gradient Descent	**0.04266**	8353	1.56
Accelerated Gradient Descent	**0.04266**	203	0.10
Naive SGD	0.05510	50000*	12.79
Decaying Learning Rate SGD	**0.04266**	14199	3.58
Newton's Method	**0.04266**	**1**	**0.006**
Adagrad	**0.04266**	42511	11.99

**Hit maximum iterations limit without fully converging*

Summary of Results for $\lambda = 1.0$

Performance comparison on the strongly regularized problem ($\lambda = 1.0$):

Optimization Method	**Loss**	**Iterations**	**Time[s]**
Naive Gradient Descent	0.16356	31209	6.39
Bounded Gradient Descent	0.16356	1768	0.32
Smooth Gradient Descent	0.16355	15	0.01
Accelerated Gradient Descent	0.16355	12	0.005
Naive SGD	0.16356	31254	7.35
Decaying Learning Rate SGD	0.16355	101	0.04
Newton's Method	0.16355	1	0.002
Adagrad	0.16356	8292	1.92

Conclusions

- **Impact of λ on Convergence:**
  Increasing λ helps optimization's algorithms to converge, making the objective function more strongly convex. A higher λ drastically reduces iterations and execution time.

- **Algorithm Performance Comparison:**
  - **Newton's Method:** Converges in exactly 1 step across all λ by leveraging second-order curvature.
  - **Accelerated GD:** Best first-order method for difficult, low-λ problems. Momentum effortlessly navigates flat valleys.
  - **Standard GD ($1/L$):** Highly efficient for well-conditioned (high-λ) problems, but struggles heavily in flat scenarios.
  - **Mini-batch SGD:** Safely converges via custom decaying rate, but stochastic noise makes strict high-precision slower than full-batch.
  - **Adagrad:** Auto-tunes step sizes without theoretical parameters, but the monotonically shrinking learning rate causes a slow late-stage crawl.